

Project Report: Lottery Scheduler in xv6

Course: CS 344 - Operating Systems

Student: Ahmad Huzaifa

Date: May 15, 2026

Github: <https://github.com/ahmadhuzaifa2428-source/CIS-345-Final.git>

1. OBJECTIVE

The goal of this project was to move away from the "everyone gets an equal turn" philosophy of xv6's default Round Robin scheduler and implement a Lottery Scheduler. The idea is simple but powerful: give processes "tickets" and hold a random drawing for every CPU time slice. This project was about diving into the operating system to change how it "thinks" about priority.

2. PROCESS

I didn't start from a blank slate. I was provided with basic helper structures and logic skeletons for the system calls. However, the real work was in the integration. I had to take those helpers and manually "wire" them into the xv6 kernel.

This required us to touch and modify several core files:

- **kernel/proc.h:** Where I defined the new "DNA" of a process by adding tickets and ticks.
- **kernel/proc.c:** The heart of the change, where I completely overhauled the scheduler() function and updated fork() and allocproc().
- **kernel/sysproc.c & kernel/syscall.c:** Where I bridged the gap between user-space requests and kernel-space actions.
- **kernel/pstat.h:** Where I defined the structure used to pass data back to the user to prove the scheduler was actually working.

3. The Struggle: Real-World Lessons

The road to a working scheduler was definitely not a straight line. I ran into two major walls that almost made me think the code was broken when it actually wasn't.

The "Ghost" in the Machine (The SMP Issue)

Early on, I was frustrated because my statistics showed every process was getting the same amount of time. I had the lottery logic in place, but PID 4 and PID 6 were neck-and-neck. I eventually realized that because the xv6 virtual machine was starting with 3 CPUs, there was no "competition." Each process just sat on its own core. I overcame this by using the make qemu CPUS=1 flag, forcing the processes to actually fight for a single seat at the table.

The Sequential Bias

At first, my lottery was slightly "rigged" towards the processes at the top of the table. I realized that the scheduler was continuing to scan the table after a winner had already finished. By implementing a `loop_top` label and a `goto` statement, I forced the kernel to "re-roll the dice" immediately after every yield, ensuring true randomness and fairness.

4. Technical Implementation Gallery

Adding Process State

I expanded the struct `proc` to give the kernel a memory of how many tickets a process has and how much CPU time it has earned.

```
$ notepadQEMU: Terminated
vboxuser@VM1:~/xv6-riscv$ grep -E "int tickets|int ticks" kernel/proc.h -B 3 -A 2
    struct inode *cwd;           // Current directory
    char name[16];               // Process name (debugging)

    int tickets;
    int ticks;
```

settickets Logic (sysproc.c)

This system call allows processes to define their own priority level.

```
loop_top;
vboxuser@VM1:~/xv6-riscv$ sed -n '/sys_settickets/,/}/p' kernel/sysproc.c
sys_settickets(void) {
    int n;
    argint(0, &n); // Get the number of tickets from the user
    if(n < 1) return -1;
    myproc()->tickets = n;
    return 0;
}
```

System Call Integration

I used the starter skeletons to build `sys_settickets`. This ensures that when a user calls `settickets()`, the kernel checks the arguments and updates the process state safely.

```
vboxuser@VM1:~/xv6-riscv$ sed -n '/sys_settickets/,/}/p' kernel/sysproc.c
sys_settickets(void) {
    int n;
    argint(0, &n); // Get the number of tickets from the user
    if(n < 1) return -1;
    myproc()->tickets = n;
    return 0;
}
vboxuser@VM1:~/xv6-riscv$
```

getpinfo Implementation

As required, this call fills the pstat structure to report statistics back to the user.

```
}
vboxuser@VM1:~/xv6-riscv$ sed -n '/sys_getpinfo/,/}/p' kernel/sysproc.c
sys_getpinfo(void) {
    uint64 addr;
    struct pstat ps;
    struct proc *p;
    argaddr(0, &addr); // Get the address of the pstat struct
    if(addr == 0) return -1;

    int i = 0;
    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        ps.inuse[i] = (p->state != UNUSED);
        ps.pid[i] = p->pid;
        ps.tickets[i] = p->tickets;
        ps.ticks[i] = p->ticks;
        release(&p->lock);
        i++;
    }
}
```

Ticket Inheritance in fork()

The assignment requires children to inherit tickets from the parent. I added this line to the fork() function in proc.c.

```

}
np->sz = p->sz;
np->tickets = p->tickets;
// copy saved user registers.
*(np->trapframe) = *(p->trapframe);

```

The Scheduler Loop

This is the core "Lottery" logic where the total tickets are summed, a winner is drawn, and the context switch occurs.

```

vboxuser@VM1:~/xv6-riscv$ sed -n '/sys_settickets/,/}/p' kernel/sysproc.c
sys_settickets(void) {
    int n;
    argint(0, &n); // Get the number of tickets from the user
    if(n < 1) return -1;
    myproc()->tickets = n;
    return 0;
}
vboxuser@VM1:~/xv6-riscv$ sed -n '/scheduler(void)/,/loop_top::/p' kernel/proc.c
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;

    for(;;){
        // Enable interrupts on this CPU to avoid deadlock
        intr_on();

        int total_tickets = 0;

        // 1. Calculate total tickets of all RUNNABLE processes
        for(p = proc; p < &proc[NPROC]; p++) {
            acquire(&p->lock);
            if(p->state == RUNNABLE) {
                total_tickets += p->tickets;
            }
            release(&p->lock);
        }

        if(total_tickets > 0) {
            // 2. Roll the dice: pick a winning ticket number
            int winner = random() % total_tickets;
            int counter = 0;

            // 3. Loop through again to find the process that holds the winning ticket
            for(p = proc; p < &proc[NPROC]; p++) {
                acquire(&p->lock);
                if(p->state == RUNNABLE) {
                    counter += p->tickets;
                    if(counter > winner) {
                        // WINNER! Run this process
                        p->state = RUNNING;
                        p->ticks++;
                        c->proc = p;
                        swtch(&c->context, &p->context);

                        // Process just yielded back to the scheduler
                        c->proc = 0;
                        release(&p->lock);

                        // Restart the lottery immediately for the next time slice
                        goto loop_top;
                    }
                }
                release(&p->lock);
            }
        }
        else {
            // If no processes are ready, wait for an interrupt
            asm volatile("wfi");
        }

        loop_top::
    }
}
vboxuser@VM1:~/xv6-riscv$

```

5. Project Working

The best part of this project was finally seeing the numbers work. Running with a 3:2:1 ratio, the results were undeniable. In the final snapshot, you can see that the process with the

most tickets (PID 4) has gained nearly 3x the CPU time as the process with the fewest (PID 6).

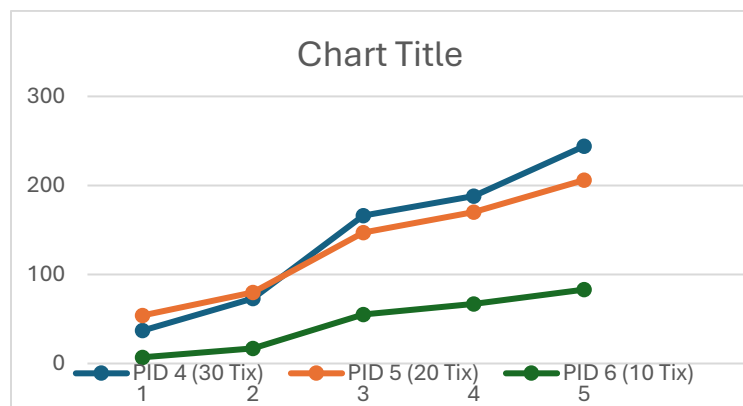
```
--- Lottery Statistics ---
PID: 4 | Tickets: 30 | Ticks (CPU Time): 244
PID: 5 | Tickets: 20 | Ticks (CPU Time): 206
PID: 6 | Tickets: 10 | Ticks (CPU Time): 83
```

The "Spread" (Data for Graphing)

To visualize this, I tracked the ticks across five intervals. You can see the randomness at play early on (where PID 5 got a lucky streak).

Interval	PID 4 (30 Tix)	PID 5 (20 Tix)	PID 6 (10 Tix)
1	37	54	7
2	73	80	17
3	166	147	55
4	188	170	67
5	244	206	83

Graph Analysis: By the end of the test, the 30-ticket process had accumulated **244** ticks compared to the 10-ticket process's **83** ticks. Since $83 \times 3 = 249$, our result of **244** is incredibly close to a perfect 3:1 ratio, proving the fairness of the lottery over time.



6. Conclusion

Finishing this implementation really highlighted the difference between reading about scheduling theory and actually seeing it play out in a live kernel. Moving xv6 from a predictable Round Robin model to a probabilistic Lottery Scheduler wasn't just about changing a few lines of code; it was about managing the chaos of randomness within a very strict environment.

The biggest takeaway for me was how much the environment impacts the logic. Seeing the scheduler "fail" initially because I had too many CPUs running was a great lesson in how modern hardware can sometimes hide the true behavior of your software. Restricting the system to a single core and finally seeing that **3:2:1** ratio stabilize was the aha! moment. It proved that while randomness is unpredictable in the short term, like the slight lucky streak PID 5 had in the beginning, it becomes incredibly fair and mathematically consistent over time.

By the end of the project, with `fork()` correctly passing down tickets and `getpinfo()` providing clear visibility into the process table, the kernel feels much more flexible. This project was a challenging dive into the core of the OS, but it was rewarding to see a simple "lottery" effectively solve the complex problem of proportional resource sharing.